Solodiscourse at Daleel 2026: Lexically-Augmented Focal-Loss Ensembling for Arabic Argumentative Discourse Unit Classification

Turab Hussain Usmani
Independent Researcher
turabofficial2003@gmail.com

Abstract

We describe our submission to Task 1 (Argumentative Discourse Unit Classification) of the Daleel 2026 shared task on Arabic argument mining, in the closed track. Given a paragraph from an Arabic editorial or debate transcript, the task is to predict the multi-label set of argumentative functions it contains, drawn from six classes: common ground, assumption, testimony, statistics, anecdote, and other. Our system fine-tunes a pretrained Arabic encoder, comparing AraBERT, MARBERT, and CAMeLBERT-mix, augmented with a small side-channel of hand-crafted lexical features motivated directly by the task's class definitions (e.g., reporting verbs for testimony, digit/percentage patterns for statistics), trained with a class-balanced focal loss and combined into a 5-fold cross-validation ensemble; CAMeLBERT-mix was selected as the final encoder for all submitted configurations. In the dev phase, our *Both*-domain configuration achieved an overall macro-F1 of 0.560 (editorial 0.435, debate 0.490). On the official test-phase Codabench leaderboard, our *Both*-domain configuration achieved an overall macro-F1 of 0.527 (editorial 0.384, debate 0.494), ranking 10th; our *Editorial*-only configuration reached an overall macro-F1 of 0.454 (editorial 0.325, debate 0.451). We report two findings grounded in these results: overall (pooled) macro-F1 consistently exceeds the macro-F1 computed on either genre subset alone, and our editorial-only model unexpectedly scores higher macro-F1 on the out-of-domain debate subset than on its own in-domain editorial subset. We discuss both as likely artifacts of per-class support sparsity at the smaller genre-subset scale rather than genuine cross-domain superiority, though we did not conduct a formal ablation study or systematic error analysis to confirm this, which we flag as the primary limitation of this report. Our code is available at https://github.com/turab-usmani/daleel-task1-closedtrack.

1 Introduction

Argument mining – the automatic identification and classification of argumentative structure in text – underlies applications ranging from opinion analysis and media literacy to explainable AI and the evaluation of language-model outputs (Stab and Gurevych, 2017). Despite this broad utility, Arabic remains comparatively under-resourced for argument mining, and in particular for discourse-level analysis that goes beyond sentence-level stance or claim detection. The Daleel 2026 shared task (Nabhani et al., 2026) addresses this gap by introducing an annotated corpus of Arabic argumentative discourse spanning two genres – editorials and debate transcripts – and defining six discourse-level argumentative functions: *common ground*, *assumption*, *testimony*, *statistics*, *anecdote*, and *other*.

We participate in **Task 1: Argumentative Discourse Unit Classification**, a paragraph-level multi-label task, under the **closed track** (provided training data, open-weight pretrained models under 70B parameters, standard NLP tooling only).

Our system strategy combines three components: (1) a pretrained Arabic transformer encoder fine-tuned as a multi-label classifier, (2) a small auxiliary feature vector of lexical cues drawn directly from the task's own class definitions (e.g., reporting verbs signal *testimony*, digit/percentage patterns and words like *dirāsa* ("study") signal *statistics*), concatenated to the pooled encoder representation before classification, and (3) a class-balanced focal loss combined with 5-fold cross-validation, which both stabilizes per-label decision-threshold tuning and yields a free ensembling gain at inference time.

Our key findings, based on the shared task's dev and test results: (1) CAMeLBERT-mix outperformed AraBERT-large and MARBERTv2 in our informal, non-ablated comparison across the

Proceedings of The Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026) - Volume 2 Shared Tasks, pages 1375–1379
October 28-29, 2026 ©2026 Association for Computational Linguistics

Split	Total	Editorial	Debate
Train	612	255	357
Dev (blind)	217	88	129
Test (blind)	213	87	126

Table 1: Number of paragraphs per split and genre (`train_task_1.jsonl`, `dev_in.jsonl`, `test_in.jsonl`). Dev and test files are distributed without gold labels; scoring is performed server-side on Codabench.

three configurations, and was used as the final encoder for all submissions; (2) pooled macro-F1 across both genres consistently exceeds the macro-F1 computed on either genre subset individually; and (3) our editorial-only configuration unexpectedly scores higher macro-F1 on the out-of-domain debate subset than on its own in-domain editorial subset. We discuss (2) and (3) in §5 and argue both are most plausibly explained by per-class support sparsity at the smaller genre-subset evaluation scale, though we did not run the ablation or error analysis needed to confirm this, which we identify explicitly as this report's main limitation rather than omit.

Code, trained-model configuration files, and inference scripts are available at `https://github.com/turab-usmani/daleel-task1-closedtrack`.

2 Background

Task setup. Task 1 requires predicting a multi-label subset $Y \subseteq \mathcal{L} = \{\text{CO}, \text{AS}, \text{TE}, \text{ST}, \text{AN}, \text{OT}\}$ for an Arabic paragraph x from editorials or debate transcripts.

Tracks. The closed track permits provided data, open-weight models under 70B parameters, and standard NLP tooling, prohibiting external labels or proprietary LLMs. We evaluate *editorial-only* and *both* configurations.

Dataset. The corpus consists of Arabic paragraphs from two genres: *editorials* (edited, formal MSA prose) and *debates* (transcribed spoken argumentation, comparatively disfluent, with dialectal features). Table 1 reports split sizes; debate is consistently larger than editorial across all splits. Systems are evaluated *in-domain* (train/test on the same genre) and *cross-domain* (train on one genre, test on the other), jointly testing multi-label classification and robustness to register shift.

Related work. Prior argument mining predominantly targets English component or premise detection (Stab and Gurevych, 2017), whereas Arabic resources focus mainly on stance and sentiment. We adapt pretrained Arabic encoders (Antoun et al., 2020; Abdul-Mageed et al., 2021) to discourse typing. While pretrained transformers excel at general semantic representation, incorporating domain-specific or task-derived lexical features via auxiliary side-channels has been widely shown to improve performance and data efficiency in low-resource and class-imbalanced classification settings. Concatenating explicit lexical markers with contextual embeddings provides an inductive bias that helps disambiguate subtle discourse boundaries under closed-track constraints.

3 System Overview

3.1 Base Model

We fine-tune a pretrained Arabic transformer as a multi-label classifier, comparing three publicly available encoders: `aubmindlab/bert-large-arabertv02` (Antoun et al., 2020), `UBC-NLP/MARBERTv2` (Abdul-Mageed et al., 2021), and `CAMeL-Lab/bert-base-arabic-camelbert-mix` (Inoue et al., 2021) (MSA, dialectal, and classical Arabic mix). Based on an informal comparison across our configurations (not a controlled ablation; §5.3), CAMeLBERT-mix was used for all final submitted results. Each paragraph is prefixed with a domain tag, `[EDITORIAL]` or `[DEBATE]`, added to the tokenizer vocabulary, so one model conditions on genre for both in-domain and cross-domain evaluation.

3.2 Lexical Feature Injection

The task's class definitions are, by design, partly lexically grounded: *testimony* is defined by attribution to an expert or authority, *statistics* by quantitative results, *anecdote* by concrete first-person experience, and *common ground* by "self-evident" framing. We hypothesize a small set of hand-crafted surface features targeting these definitions supplies signal that is hard for an encoder to reliably recover from a few hundred examples, and can be injected essentially for free.

To construct the keyword lists, we manually extracted high-precision indicator terms and discourse markers directly from the official task definition guidelines (such as explicit reporting verbs for testimony and epistemic modals for opinion), supplemented by inspecting high-frequency unigrams and bigrams characteristic of each class in the train-

Category	Representative cues (transliterated)
Reporting	*qāla*, *ṣarraḥa*, *afāda*, *bi-ḥasab*
Study/stats	*dirāsa*, *iḥṣā'iyya*, *istiṭlā'*
Modal/opinion	*yajibu*, *a'taqidu*, *min al-murajjaḥ*
Common ground	*min al-ma'rūf*, *lā shakka anna*
Anecdote	*atadhakkaru*, *jarrabtu*, *bi-nafsī*
Meta-discourse	*al-miḥwar al-awwal*, *sa-uwaḍḍiḥu*

Table 2: Representative keyword categories used for lexical feature extraction (§3.2). Keywords are matched in the original Arabic script in code; transliterations are shown here for readability given the pdfLaTeX compilation pipeline.

ing split.

For a paragraph with word tokens $w_1, \ldots, w_n$, we compute $\mathbf{f} \in \mathbb{R}^{10}$: six keyword-density features (occurrences per 100 words of curated Arabic keyword lists for *reporting*, *study/statistics*, *modal/opinion*, *common-knowledge*, *anecdotal*, and *meta-discourse* markers, e.g. "the first axis"), a percent/digit binary indicator pair, a quotation-mark density, and $\log(1+n)$. Table 2 lists representative keywords.

Let $h_{\text{CLS}} \in \mathbb{R}^d$ be the pooled `[CLS]` representation. We project $\mathbf{f}$ via $g(\mathbf{f}) = \text{ReLU}(W_1\mathbf{f} + b_1) \in \mathbb{R}^{32}$ and classify from the concatenation:

$$\hat{y} = \sigma\big(W_2\,[h_{\text{CLS}}\,;\,g(\mathbf{f})] + b_2\big) \in [0,1]^{|\mathcal{L}|}. \quad (1)$$

3.3 Class-Balanced Focal Loss

Due to severe label imbalance across the six classes and the substantial variation in classification difficulty between frequent and rare argumentative functions, standard binary cross-entropy can lead to degraded performance on underrepresented labels. We instead employ focal loss (Lin et al., 2017) with a *capped* class weight to prevent runaway over-prediction:

$$\alpha_c = \frac{\min\big(\max(1,\ \frac{1/\sqrt{\hat{p}_c}}{\min_{c'} 1/\sqrt{\hat{p}_{c'}}}),\ 3\big)}{\min\big(\ldots\big) + 1}, \quad (2)$$

where $\hat{p}_c$ is the empirical frequency of class c, and the per-example loss is

$$\mathcal{L} = -\frac{1}{|\mathcal{L}|}\sum_c \alpha_{c,y}(1-p_t)^\gamma \log p_t, \quad (3)$$

with $p_t = \hat{y}_c$ if $y_c{=}1$ else $1{-}\hat{y}_c$, $\alpha_{c,y} = \alpha_c$ if $y_c{=}1$ else $1{-}\alpha_c$, and $\gamma{=}2$. Capping the weight at $3\times$ retains the intended down-weighting of easy majority-class examples (via the focal term $(1-p_t)^\gamma$) while preventing runaway over-prediction of rare classes.

3.4 Cross-Validated Threshold Tuning and Ensembling

Rather than a single noisy held-out split, we train with 5-fold cross-validation and concatenate out-of-fold (OOF) predicted probabilities across folds to get estimates for the entire training set. We tune a per-class threshold τ_c by grid search over $\{0.05, \ldots, 0.95\}$ to directly maximize per-class F1 on these OOF predictions, since the task is F1-scored and a blind 0.5 cutoff is not calibrated to that objective. At inference, we average the sigmoid outputs of all 5 fold checkpoints (bagging) before applying τ_c; if no class clears its threshold, we fall back to the single highest-probability class, so the empty-set prediction never occurs.

3.5 Configurations

We evaluate two training settings: **Editorial** (editorial paragraphs only) and **Both** (the union, with the domain tag above). Both share the architecture and training procedure of this section, differing only in training data.

4 Experimental Setup

Data splits. We use the organizer-provided `train.jsonl` for both model fitting and, via 5-fold cross-validation (`sklearn.model_selection.KFold`, shuffled, seed 42), for threshold tuning, since the organizer-provided `dev_in.jsonl` file is distributed without gold labels (evaluation is performed server-side on Codabench). Final predictions are produced by ensembling the 5 fold checkpoints and evaluated on both the dev-phase and official test-phase blind splits.

Preprocessing. We strip diacritics and tatweel and collapse whitespace, avoiding aggressive normalization to preserve subword tokenizer alignment (max length 256 tokens).

Hyperparameters. We explored several hyperparameter combinations during development; we report the configuration used for our final submitted results. We fine-tune each fold for up to 12 epochs (batch size 8, gradient accumulation 2; AdamW, lr 2×10^{-5}, 10% warmup, weight decay 0.01, fp16, gradient checkpointing to fit 16GB VRAM), retaining the best-macro-F1 epoch per fold rather than the last. Focal loss uses $\gamma = 2$ with the capped class weight of §3.3. Full hyperparameters are in Appendix A (Table 4).

Phase	Config	Overall	Edit.	Deb.	Rank
Dev	Both	0.560	0.435	0.490	–
Test	Both	0.527	0.384	0.494	10
	Editorial	0.454	0.325	0.451	–

Table 3: Dev-phase and official test-phase Codabench leaderboard macro-F1 scores (per official `scoring.py`): *Overall* pools all instances; *Edit./Deb.* evaluate editorial/debate subsets.

Tools and models. Built with PyTorch 2.11, HuggingFace `transformers` 5.13.1 (Wolf et al., 2020), and `scikit-learn` 1.6.1. We compare `aubmindlab/bert-large-arabertv02` (Antoun et al., 2020), `UBC-NLP/MARBERTv2` (Abdul-Mageed et al., 2021), and select `CAMeL-Lab/bert-base-arabic-camelbert-mix` (Inoue et al., 2021).

Metrics. Following the official evaluation script, we report macro-F1 over the six labels computed with `scikit-learn` (`precision_recall_fscore_support`) jointly over both genres and separately per genre (editorial-only, debate-only), matching the organizers' `scoring.py`.

5 Results

5.1 Leaderboard Results

Table 3 reports our validation results from the development phase alongside official evaluation results on the Daleel 2026 test-phase Codabench leaderboard, computed server-side by the organizers' `scoring.py`. The evaluation script computes macro-F1 three times per submission: pooling all instances (*Overall*), and separately on the editorial-only and debate-only subsets.

In the development phase, our *Both*-domain configuration achieved an overall macro-F1 of 0.560 (0.435 on editorial, 0.490 on debate). On the official blind test set, our *Both*-domain submission achieved an overall macro-F1 of 0.527 (0.384 on editorial, 0.494 on debate), securing 10th place on the official task leaderboard. Our test-phase *Editorial*-only configuration achieved an overall macro-F1 of 0.454 (0.325 on editorial, 0.451 on debate).

5.2 Analysis

Two patterns in Table 3 are counter-intuitive; we discuss both despite lacking the ablation and per-class breakdown needed to fully explain them (§5.3).

Pooled macro-F1 exceeds either subset's macro-F1. Overall macro-F1 consistently exceeds individual editorial and debate scores across dev and test phases. This is hypothesized to be an artifact of small subset sizes (87–129 instances), where rare classes with sparse support disproportionately drag down unweighted macro-F1, whereas pooling genres increases support and smooths the metric—though confirming this requires an uncomputed per-class breakdown.

Editorial-only training scores higher out-of-domain. On the test set, the *Editorial* model scored higher on out-of-domain debates (0.451) than in-domain editorials (0.325). This is likely caused by the smaller editorial split size (255 train / 87 test paragraphs), leading to rarer class underfitting and heightened macro-F1 penalty from support sparsity.

5.3 Limitations of This Analysis

We did not conduct a controlled ablation (isolating the lexical feature side-channel, capped focal loss, or CV ensembling of §3) or a systematic error analysis (confusion matrices, categorized examples), due to time constraints. The CAMeLBERT-mix vs. AraBERT vs. MARBERT comparison in §4 was likewise an informal development-time choice, not a controlled comparison. Completing both is the most direct next step (§6).

6 Conclusion

We presented our Daleel 2026 Task 1 submission: fine-tuned CAMeLBERT-mix with definition-derived lexical features, capped focal loss, and 5-fold CV threshold ensembling, achieving 0.527 test macro-F1 (10th place). We observed two evaluation-scale anomalies (pooled macro-F1 superiority and inverted domain performance), which likely stem from per-class support sparsity in small genre subsets.

Limitations. We did not run a controlled ablation or systematic error analysis (§5.3), so the design rationale for the lexical features and capped focal loss (§3) reflects motivating hypotheses, not measured contributions. Keyword lists were built manually and likely under-cover dialectal debate register. We did not explore open-weight instruction-tuned LLMs (permitted up to 70B parameters).

Future work. The immediate next step is the ablation and error analysis flagged above, particularly a per-class precision/recall breakdown to test the support-sparsity explanation in §5.2. Further directions: sentence/clause-level auxiliary supervi-

sion aggregated to paragraph level; ensembling CAMeLBERT-mix with AraBERT/MARBERT; and, once error analysis identifies the most-confused label pairs, checking whether that reflects an inter-annotator-agreement ceiling.

References

Muhammad Abdul-Mageed, AbdelRahim Elmadany, and El Moatez Billah Nagoudi. 2021. ARBERT and MARBERT: Deep bidirectional transformers for Arabic. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 7088–7105.

Wissam Antoun, Fady Baly, and Hazem Hajj. 2020. AraBERT: Transformer-based model for Arabic language understanding. In *Proceedings of the 4th Workshop on Open-Source Arabic Corpora and Processing Tools (OSACT)*, pages 9–15.

Go Inoue, Bashar Alhafni, Nurpeiis Baimukan, Houda Bouamor, and Nizar Habash. 2021. The interplay of variant, size, and task type in Arabic pre-trained language models. In *Proceedings of the Sixth Arabic Natural Language Processing Workshop*, pages 92–104.

Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. 2017. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2980–2988.

Sara Nabhani, Nahla Bassyouni, Ali Al-Zawqari, Mohammad Khader, and Khalid Al-Khatib. 2026. Daleel 2026 shared task on arabic argumentative discourse mining. Budapest, Hungary. Association for Computational Linguistics.

Christian Stab and Iryna Gurevych. 2017. Parsing argumentation structures in persuasive essays. *Computational Linguistics*, 43(3):619–662.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, and 3 others. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45.

A Implementation Details

Table 4 lists the complete hyperparameter settings and hardware configuration used for all training folds.

Hyperparameter	**Value**
Base encoder	CAMeLBERT-mix
Max sequence length	256 subwords
Batch size (per device)	8
Gradient accumulation	2 (effective 16)
Epochs (max, per fold)	12
Learning rate	2×10^{-5}
Warmup ratio	0.10
Weight decay	0.01
Optimizer	AdamW
Precision	fp16
Focal loss γ	2.0
Max class weight cap	$3\times$
Lexical feature projection dim	32
Cross-validation folds	5
Threshold grid	$\{0.05, 0.10, \ldots, 0.95\}$
Random seed	42
GPU	$1\times$ NVIDIA T4 (16GB)

Table 4: Full hyperparameter configuration.

B Lexical Feature Keyword Lists

Table 5 reproduces the complete keyword lists used for lexical feature extraction (§3.2), shown here transliterated for readability; matching is performed against the original Arabic-script strings in code.

Category	**Keywords (transliterated)**
Reporting	qāla, ṣarraḥa, akkada, a'lana, ḥasab, wafqan li, bi-ḥasab, dhakara, afāda, naqala, awḍaḥa, shaddada
Study/stats	dirāsa, dirāsāt, baḥth 'ilmī, iḥṣā'iyya, iḥṣā', istiṭlā', bayānāt, taqrīr ṣādir, nisba
Modal/opinion	yajibu, yanbaghī, min al-mafrūḍ, a'taqidu, fī ra'yī, min al-wāḍiḥ, yabdū anna, min al-murajjaḥ, arā anna, min al-ḍarūrī
Common knowledge	min al-ma'rūf, min al-badīhī, lā yakhfā 'alā aḥad, badīhī, min al-musallam bihi, kamā huwa ma'rūf, lā shakka anna
Anecdote	atadhakkaru, jarrabtu, shahidtu, 'ishtu, ra'aytu, ḥadatha ma'ī, qiṣṣatī, tajribatī, bi-nafsī, ḥaṣala ma'ī
Meta-discourse	al-miḥwar al-awwal, al-miḥwar al-thānī, al-nuqṭa al-ūlā, al-nuqṭa al-thāniya, sa-uwaḍḍiḥu, sa-ataḥaddathu, da'ūnī, kamā dhakartu, bidāya, akhīran

Table 5: Complete keyword lists per category used for lexical feature extraction (§3.2). Keywords are counted as substring occurrences per 100 words of paragraph text.